

Sprawozdanie z projektu Deal&Meal;

1. Cel projektu

Celem projektu było przygotowanie prostej aplikacji typu fullstack na podstawie projektu UI/UX wykonanego w Figmie. Aplikacja została zaprojektowana jako system wspierający planowanie posiłków, wyszukiwanie przepisów promocyjnych oraz tworzenie listy zakupów. Projekt łączy warstwę frontendową odpowiedzialną za interfejs użytkownika z warstwą backendową obsługującą dane, logikę biznesową i komunikację z bazą danych.

Nazwa aplikacji: Deal&Meal;

Główna idea systemu:

- prezentacja przepisów dostępnych w promocji,
- zapis preferencji żywieniowych użytkownika,
- podgląd składników przepisu,
- podmiana produktu na tańszy zamiennik,
- wygenerowanie listy zakupów.

2. Zastosowane technologie

W projekcie wykorzystano następujące technologie:

Frontend

- React

Frontend został napisany w bibliotece React. Pozwala ona budować interfejs użytkownika z wykorzystaniem komponentów i aktualizować widok dynamicznie po zmianie stanu aplikacji.

- JavaScript

Logika po stronie klienta została przygotowana w języku JavaScript.

- Vite

Narzędzie Vite zostało użyte do uruchamiania i budowania aplikacji frontendowej. Zapewnia szybki start projektu i prostą konfigurację środowiska developerskiego.

- CSS

Stylowanie interfejsu wykonano w osobnym pliku CSS. Odpowiada ono za układ aplikacji, kolory, typografię oraz wygląd kart, formularzy i sekcji.

Backend

- Python

Warstwa serwerowa została przygotowana w Pythonie.

- FastAPI

Do budowy API wykorzystano framework FastAPI. Ułatwia on tworzenie endpointów REST, walidację danych i generowanie dokumentacji API.

- SQLAlchemy

SQLAlchemy zostało użyte jako warstwa ORM. Pozwala ono odwzorować obiekty Pythona na rekordy w bazie danych.

- Pydantic

Biblioteka Pydantic służy do definiowania schematów danych wejściowych i wyjściowych dla API.

Baza danych

- SQLite

W finalnej wersji projektu zastosowano SQLite, ponieważ jest to najprostsza baza do uruchomienia lokalnego. Nie wymaga osobnej instalacji serwera i zapisuje dane do jednego pliku bazy.

3. Zastosowanie programowania obiektowego

W projekcie wykorzystano podejście obiektowe głównie po stronie backendu.

Modele obiektowe

W folderze backend/app/models.py zostały utworzone klasy reprezentujące główne encje systemu:

- Preference
- Recipe
- Ingredient
- Replacement

Każda z tych klas opisuje obiekt domenowy aplikacji i odpowiada tabeli w bazie danych.

Przykład podejścia obiektowego:

- obiekt Recipe przechowuje dane przepisu,
- obiekt Ingredient opisuje składnik przypisany do konkretnego przepisu,
- obiekt Replacement opisuje alternatywny produkt dla składnika,
- obiekt Preference przechowuje ustawienia użytkownika.

Repozytoria

W pliku `backend/app/repositories.py` zostały wydzielone klasy:

- `PreferenceRepository`
- `RecipeRepository`

Ich zadaniem jest odseparowanie logiki dostępu do danych od reszty aplikacji. Dzięki temu kod jest bardziej uporządkowany i łatwiejszy w rozwijaniu.

Serwisy

W pliku `backend/app/services.py` zostały przygotowane klasy:

- `DashboardService`
- `PreferenceService`
- `RecipeService`
- `ShoppingListBuilder`

To właśnie tutaj znajduje się logika biznesowa aplikacji. Jest to zgodne z podejściem obiektowym, ponieważ:

- każda klasa ma określoną odpowiedzialność,
- dane i operacje są grupowane tematycznie,
- aplikacja jest podzielona na warstwy.

Korzyści z OOP w tym projekcie

- czytelniejsza struktura kodu,
- prostsze dodawanie nowych funkcji,
- oddzielenie logiki interfejsu od logiki danych,
- możliwość łatwej rozbudowy projektu.

4. Architektura aplikacji

Projekt został podzielony na dwie główne części:

Frontend

Frontend odpowiada za:

- wyświetlanie widoków,
- pobieranie danych z backendu,

- obsługę kliknięć użytkownika,
- zapisywanie zmian preferencji,
- prezentowanie listy zakupów i przepisów.

Backend

Backend odpowiada za:

- wystawianie endpointów API,
- dostęp do bazy danych,
- tworzenie danych startowych,
- zwracanie danych do widoków frontendowych,
- obsługę logiki podmiiany składników.

Baza danych

Baza danych przechowuje:

- preferencje użytkownika,
- przepisy,
- składniki,
- możliwe zamienniki składników.

5. Opis komunikacji Frontend-Backend

Komunikacja między frontendem i backendem odbywa się przez HTTP z użyciem formatu JSON.

Frontend korzysta z pliku `src/api.js`, w którym zdefiniowano funkcje odpowiedzialne za wysyłanie zapytań do API.

Adres backendu:

`http://127.0.0.1:8000`

Schemat działania:

1. Użytkownik otwiera aplikację frontendową.
2. React wykonuje zapytania do backendu.
3. Backend przetwarza żądania.
4. Dane są pobierane z bazy SQLite.
5. Backend zwraca odpowiedź JSON.
6. Frontend aktualizuje widok na podstawie otrzymanych danych.

Przykładowe operacje komunikacji

- pobranie danych ekranu głównego,
- pobranie preferencji użytkownika,
- zapis preferencji użytkownika,
- pobranie listy przepisów,
- pobranie szczegółów przepisu,
- podmiana składnika w przepisie,
- pobranie listy zakupów.

6. Dokumentacja API

W backendzie przygotowano następujące endpointy:

GET /api/dashboard

Opis:

Zwraca dane ekranu głównego, czyli polecany przepis, listę rekomendowanych przepisów oraz podstawowe statystyki.

Przykładowe dane:

- przepis promocyjny,
- lista polecanych przepisów,
- kalorie,
- waga,
- oszczędności.

GET /api/preferences

Opis:

Zwraca aktualne preferencje użytkownika zapisane w bazie danych.

PUT /api/preferences

Opis:

Zapisuje zmienione preferencje użytkownika.

Przykładowe pola:

- age
- weight

- height
- goal
- diet_type
- allergies
- shopping_mode

GET /api/recipes

Opis:

Zwraca listę dostępnych przepisów.

GET /api/recipes/{recipe_id}

Opis:

Zwraca szczegóły wybranego przepisu, w tym składniki i możliwe zamienniki.

GET /api/shopping-list

Opis:

Zwraca listę zakupów przygotowaną dla wybranego przepisu.

POST /api/recipes/{recipe_id}/swap

Opis:

Pozwala zamienić składnik przepisu na wybrany zamiennik, a następnie zwraca zaktualizowany przepis i listę zakupów.

Przykładowe body:

```
{  
  "ingredient_id": 1,  
  "replacement_id": 2  
}
```

Przykładowa odpowiedź:

- zaktualizowany przepis,
- nowa lista zakupów.

7. Opis UX i działania aplikacji

Aplikacja została zaprojektowana jako prosty system mobilny wspierający użytkownika w wyborze posiłków oraz w planowaniu zakupów.

Co widzi użytkownik

Użytkownik w projekcie może zobaczyć następujące widoki:

1. Ekran rejestracji

Użytkownik może założyć nowe konto, podając imię i nazwisko, adres e-mail i hasło.

2. Ekran logowania

Użytkownik loguje się do istniejącego konta.

3. Ekran resetu hasła

Użytkownik może podać adres e-mail i otrzymać link do resetu hasła.

4. Ekran ulubionych produktów

Użytkownik wybiera produkty, które lubi jeść.

5. Ekran preferencji zakupów

Użytkownik wybiera, czy chce kupować w jednym sklepie czy korzystać z kilku sklepów dla większych oszczędności.

6. Ekran preferencji żywieniowych

Użytkownik ustawia wiek, wagę, wzrost, cel, typ diety oraz alergie.

7. Ekran główny

Na ekranie głównym widoczne są statystyki, szybkie akcje oraz polecane przepisy promocyjne.

8. Ekran przepisu promocyjnego

Użytkownik widzi danie, czas przygotowania, koszt i składniki.

9. Ekran listy składników

Użytkownik może sprawdzić, czego potrzebuje do przygotowania dania.

10. Ekran zamiany produktu

Użytkownik może wymienić składnik na tańszy odpowiednik z promocji.

11. Ekran listy zakupów

Użytkownik widzi finalną listę produktów, sklep i przewidywane oszczędności.

12. Ekran lokalizacji sklepu

Użytkownik może zobaczyć odległość i czas dojścia do sklepu.

Założenia UX

- prosty i czytelny interfejs,
- duże sekcje kartowe,
- skupienie na oszczędności czasu i pieniędzy,
- szybki dostęp do najważniejszych funkcji,
- prowadzenie użytkownika od wyboru przepisu do gotowej listy zakupów.

8. Podział na komponenty frontendowe

Frontend został zorganizowany w postaci komponentów React.

Główny komponent

- App

Odpowiada za:

- przechowywanie stanu aplikacji,
- pobieranie danych z API,
- przełączanie ekranów,
- obsługę akcji użytkownika.

Komponenty widoków

- HomeScreen

Widok ekranu głównego z przepisem promocyjnym i listą polecanych przepisów.

- PreferencesScreen

Widok formularza preferencji użytkownika.

- RecipesScreen

Widok szczegółów przepisu i składników.

- ShoppingScreen

Widok listy zakupów.

Komponent pomocniczy

- StatCard

Mały komponent do wyświetlania statystyk na ekranie głównym.

Pozostałe pliki frontendu

- src/api.js

Zawiera funkcje do komunikacji z backendem.

- src/styles.css

Zawiera stylowanie aplikacji.

- src/main.jsx

Odpowiada za uruchomienie aplikacji React.

9. Podział backendu na warstwy

Backend został podzielony na kilka warstw:

Warstwa startowa

- backend/app/main.py

Definiuje aplikację FastAPI, konfigurację CORS oraz endpointy API.

Warstwa bazy danych

- backend/app/database.py

Zawiera konfigurację bazy danych SQLite oraz sesji SQLAlchemy.

Warstwa modeli

- backend/app/models.py

Definiuje obiekty ORM i strukturę tabel.

Warstwa schematów

- backend/app/schemas.py

Definiuje schematy wejścia i wyjścia API.

Warstwa repozytoriów

- backend/app/repositories.py

Odpowiada za pobieranie i zapisywanie danych w bazie.

Warstwa serwisów

- backend/app/services.py

Odpowiada za logikę biznesową aplikacji.

Dane startowe

- backend/app/seed.py

Tworzy przykładowe dane po uruchomieniu aplikacji.

10. Zrzuty ekranu

W projekcie źródłem wyglądu były ekrany przygotowane w Figmie i przekazane w formie PDF. Do sprawozdania można dołączyć następujące zrzuty:

1. Rejestracja

Źródło: iPhone 16 Pro - 1.pdf

2. Logowanie

Źródło: iPhone 16 Pro - 2.pdf

3. Reset hasła

Źródło: iPhone 16 Pro - 3.pdf

4. Ulubione produkty

Źródło: iPhone 16 Pro - 5.pdf

5. Opcje zakupów

Źródło: iPhone 16 Pro - 6.pdf

6. Lista zakupów

Źródło: iPhone 16 Pro - 7.pdf

7. Lokalizacja sklepu

Źródło: iPhone 16 Pro - 8.pdf

8. Preferencje

Źródło: iPhone 16 Pro - 11.pdf

9. Posiłek promocyjny

Źródło: iPhone 16 Pro - 12.pdf

10. Ekran główny

Źródło: iPhone 16 Pro - 13.pdf

11. Lista składników

Źródło: iPhone 16 Pro - 14.pdf

12. Wyszukiwarka przepisów

Źródło: iPhone 16 Pro - 15.pdf

13. Szczegóły przepisu

Źródło: iPhone 16 Pro - 16.pdf

14. Zamiana produktu

Źródło: iPhone 16 Pro - 17.pdf

15. Zapisane preferencje

Źródło: iPhone 16 Pro - 18.pdf

16. Zaktualizowana lista zakupów

Źródło: iPhone 16 Pro - 19.pdf

11. Wnioski końcowe

Projekt Deal&Meal; pokazuje, w jaki sposób można połączyć projekt UI/UX z prostą aplikacją fullstack. Została zachowana podstawowa logika przepływu użytkownika od ustawienia preferencji, przez wybór przepisu, aż do wygenerowania listy zakupów.

Najważniejsze elementy projektu:

- zastosowanie komponentowego frontendu w React,
- przygotowanie backendu REST w FastAPI,
- użycie bazy SQLite dla prostego uruchomienia lokalnego,
- wykorzystanie podejścia obiektowego po stronie backendu,
- odwzorowanie głównych ekranów i przepływów z projektu graficznego.

Projekt może być dalej rozwijany między innymi o:

- prawdziwe logowanie użytkowników,
- większą liczbę przepisów,
- integrację z mapami,
- filtrowanie wyników,
- zapisywanie historii zakupów.